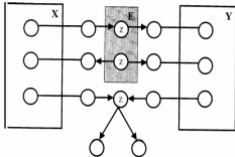


Chain rule: $p(x, y) = p(x)p(y x) = p(y)p(x y)$				
Extended chain rule: $p(x, y, z, w) = p(x)p(y x)p(z x, y)p(w x, y, z)$				
Conditional probability as a distribution: $\sum_x p(X Y = y) = 1$				
Independence: If $p(X Y) = p(X)$, then X and Y are independent for all values of X				
Conditional independence: $X \perp Y Z \leftrightarrow p(X, Y Z) = p(X Z)p(Y Z)$				
Linear Programming: Set of real-valued variables, linear function with minimization/maximization objective, and a set of linear constraints Guaranteed <i>convex function</i> , can find a global optimum				
Factored joint distribution of a Bayes net: $p(x_1, \dots, x_n) = \prod p(x_i \text{parents}(x_i))$				
Local Markov property: $X_i \perp \text{NonDescendants}(X_i) \text{Parents}(X_i)$				
Trail blocking: Evidence in cascade , evidence in common cause , or no evidence in V structure (including descendants)				
D-separation: when all trails between 2 sets of nodes are blocked given evidence set. D-separation implies conditional independence 				
Cascade $X \rightarrow E \rightarrow Y$ Common cause $X \leftarrow E \rightarrow Y$ V structure $X \rightarrow \neg(E \text{ and descendants}) \leftarrow Y$				
Bayes net stores joint probabilities of nodes and immediate parents. Sum product variable elimination exploits conditional independences in a Bayes net to compute joint probabilities without a full joint distribution table.				
Search problem: Comprises <i>state space, operators, costs, objective</i>				
Search	Informed?	Metric	Queue replace?	Explore replace?
BFS	No	Breadth, node ordering	No	No
DFS	No	Depth, node ordering	No	No
BFS opt	No	Path cost $g(n)$	Yes	No
Best-first	Yes	Heuristic $h(n)$	Yes	No
A*	Yes	Path and heuristic $g(n) + h(n)$	Yes	Yes
Admissible heuristic: $\forall$ states $n, h(n) \leq c(s, g)$, i.e. heuristic does not exceed min cost				
Consistent heuristic: $\forall$ states $n, p, h(n) \leq c(n, p) + h(p)$ (triangle inequality)				
Markov Random Fields capture correlation (undirected edges) between variables, joint distribution is the product of <i>clique potentials</i> and must be normalized				
MPE (most probable explanation) achieved by variable elimination with a forward and backward pass. Given variables A , maximize $P(B E = e)$ for $B = A \setminus E$				
Approximate inference: Stochastically choose from parent to leaf nodes, count frequency of values				
Likelihood weighting: When using approximate inference with evidence observed, weight each evidence node by $P(X_i = e \text{parents})$ to compensate				
MDP terms: (S, A, P, R) tuple with states S , actions A , transitions P and rewards R . <u>Decision epoch</u> -finite/infinite horizon, <u>absorbing state</u> -transition outside is impossible, <u>markov assumption</u> - <u>past</u> $\perp$ <u>future</u> <u>present</u> , <u>transition function</u> must be normalized $\sum_{s'} P(s' s, a) = 1$				
EU defined for every state and action: $Q(s, a) = \sum_{s'} P(s' s, a) [R(s, a, s') + V(s')]$				
MEU defined for every state: $V = \max_a Q(s, a)$				
Policy can be <i>stationary/non-stationary, deterministic/stochastic</i>				
Value iteration: Set $V^0(s) = 0 \forall$ states s , each time step t , update V s: $Q^t(s, a) = \sum_{s'} P(s' s, a) [R(s, a, s') + \gamma V^{t-1}(s')]$, where discount factor $\gamma < 1$ $V^t(s) = \max_a Q^t(s, a)$				
Convergence when $\max_s V^{t+1}(s) - V^t(s) < \epsilon$				
Optimal policy guarantee: $\pi^*(s) = \arg \max_a Q^{(t+1)}(s, a)$				
Policy iteration: Start with a random policy and make improvements over time. $V^\pi(s) = Q(s, \pi(s))$, for each state s , if $Q(s, \hat{a}) > Q(s, \pi(s))$ then $\pi(s) = \arg \max_a \sum_{s'} P(s' s, a) [R(s, a, s') + \gamma V(s')]$ yep				
Linear Programming: Minimize $\sum_s V(s)$ s.t. $V(s) \geq \sum_{s'} P(s' s, a) [R(s, a, s') + \gamma V(s')], \forall a$				
Multi-armed bandit: Given tries t , action count $k_t(a)$, action rewards $R_1(a), \dots, R_{k_t}(a)$, expected payoff $Q_t(a) = \frac{R_1(a) + \dots + R_{k_t}(a)}{k_t(a)}$				

Greedy: $\arg \max_a Q_t(a)$	ϵ-greedy: $\arg \max_a Q_t(a)$ (1- ϵ) of the time else random
Softmax: $P(a) = \frac{e^{Q_t(a)}}{\sum_b e^{Q_t(b)}}$	Upper confidence bound: $\arg \max_a (Q_t(a) + c \sqrt{\frac{\log(t)}{k_t(a)}})$, with constant c

Bellman Equation: $Q_{t+1}(a) = Q_t(a) + \frac{1}{k_t(a)} [R_{k_t(a)}(a) - Q_t(a)]$

Q-learning: World is a set of *discrete and finite* states and actions. An **experience** is (s_t, a_t, r_t, s_{t+1}) where s_t is **state** at time step t , a_t is **action**, r_t is **reward** and s_{t+1} is **new state**. An **episode** is a sequence of experiences from start to a **terminal state**

Tabular Q-learning: Store $Q(s,a)$ for each state and action in 2D array (rows are states and columns are actions). On new experience, compute $Q^{new}(s_t, a_t) = Q(s_t, a_t) + \alpha [r_t + \gamma \cdot \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$, where α is step size/learning rate, and γ is discount factor. Repeat for a fixed number of experiences, or upon some convergence condition (among $Q(s, a)$ or $V(s)$)

Feature-based learning: Instead of states, learn specific features. Represent Q value according to weighted combination of these features $\Rightarrow Q(s, a) = w_1 \cdot f_1(s, a) + \dots + w_n \cdot f_n(s, a)$, with weights w_i and features f_i

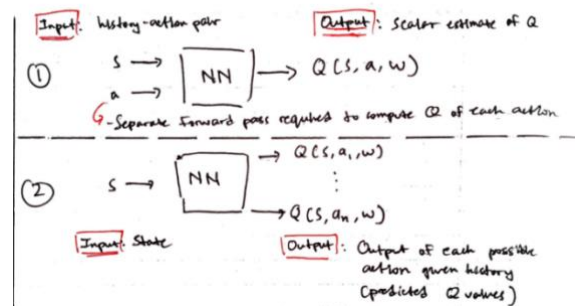
Activation functions: Sigmoid $f(x) = \frac{1}{1+e^{-x}}$, tanh $f(x) = \tanh(x)$, ReLu (rectified linear unit) $f(x) = \max(0, x)$ – nonlinear functions applied to values computed by neurons to improve usefulness of the model

DQN: Value function approximated by deep neural network, update using MSE loss with gradient descent

Experience replay – Randomly sample transitions from replay memory to remove correlations between states

Fixed targets – freeze target NN for a fixed number of steps C before updating

2 threads – 1 thread acts in environment according to ϵ -greedy, 1 thread retrieves minibatches from replay buffer



Distributed Constraint Optimization (DCOP): Given agents A , variables V , domain values D , constraints F , mapping of variables to agents a with a minimizing/maximizing assignment

ADOPT: Transform constraint graph into pseudo-tree. Starting from root node, send value (variable assignment) to child nodes and propagate costs to parent nodes. Update threshold(lower bound) with every level explored, update upper bound when deepest node has been explored.

DPOP: Transform constraint graph into pseudo-tree. Starting from leaf nodes, propagate costs to parent nodes. Once parent receives all costs from children, it chooses best value and informs children. Propagate back down.

k-optimality: No deviation by $\leq k$ agents in the solution can decrease the solution cost. Form a grp of $\leq k$ agents, choose best values within the grp and send to neighbours. Repeat until no improvements

Algo	Sol. quality	Memory/agent	Message size	No. messages
ADOPT	Optimal	$O(n)$	$O(n)$	$O(2^n)$
DPOP	Optimal	$O(2^n)$	$O(2^n)$	$O(n)$
k-optimal	Suboptimal	$O(n)$	$O(n)$	$O(2^n)$

Nash equilibrium: No player has incentive to change. every finite game has ≤ 1 nash equilibrium, pure or mixed

Fictitious play: Each agent begins with random strategy, computes best response against aggregated belief of opponents actions. Converges for potential, identical interest, zero sum gaes

Potential function ϕ : Defined such that $\phi(a'_i, a_{-i}) - \phi(a''_i, a_{-i}) = u_i(a'_i, a_{-i}) - u_i(a''_i, a_{-i})$, where a'_i, a''_i are possible actions the agent can take, and a_{-i} are actions of other agents

Correlated equilibrium: Given a probability distribution on joint strategies, no agent has any incentive to deviate. Let s_i be the recommended strategy for player i then we have $\sum_{s_{-i}} P(s_i, s_{-i}) u_{s_i, s_{-i}}^i \geq \sum_{s_{-i}} P(s_{i\#}, s_{-i}) u_{s_{i\#}, s_{-i}}^i$
 $\forall s_{i\#} \in S_i, \forall s_{-i} \in S_{-i}, \forall i$

Stackelberg game: Leader policy $x \in X$, follower policy $q \in Q$.

Follower LP: $\max_q \sum_{j \in Q} \sum_{i \in X} C_{ij} x_i q_j$ **S.T.** $\sum_{j \in Q} q_j = 1, 0 \leq q_j \leq 1$

Follower dual: $\min_a$ **S.T.** $a \geq \sum_{i \in X} C_{ij} x_i$

Leader LP: $\max_x \sum_{i \in X} \sum_{j \in Q} R_{ij} q_j(x) x_i$ **S.T.** $\sum_{i \in X} x_i = 1, 0 \leq x_i \leq 1$

Combining with dual: $\max_{x, q, a} \sum_{i \in X} \sum_{j \in Q} R_{ij} x_i q_j$ **S.T.** $\sum x_i = 1, \sum q_j = 1, 0 \leq a - \sum_{i \in X} C_{ij} x_i \leq (1 - q_j)M$

